Unidad 4 bis Transacciones

¿Que es una transacción?

Es un conjunto de operaciones que forman una unidad lógica de trabajo , la cual debe ejecutarse por completo o no ejecutarse en absoluto, garantizando la integridad de los datos(atomicidad).

Propiedades ACID (Atomicity, Consistency, Isolation and Durability).

1. **Atomicidad: Dice que todas las operaciones deben realizarse correctamente o ninguna, si ocurre un fallo se revierte todo, la gestiona el componente de gestión de transacciones**

2. **Consistencia: se mantiene la coherencia de los datos, es responsabilidad del programador asegurar la consistencia**

3. **Aislamiento: el sistema garantiza que una transacción haya terminado su ejecución antes de que otra transacción comience su ejecución evitando lecturas parciales, es controlado por el componente de control de concurrencia**

4. **Durabilidad: una vez completada la transacción, los cambios persisten en el sistema aun si falla, se encarga el gestor de integridad o de recuperación.**

Control de concurrencia

Permite que varias transacciones se ejecuten a la vez sin generar errores, los más comunes son los protocolos de bloqueo, los esquemas de ordenación por marcas temporales, las técnicas de validación y los esquemas multiversión.

Esquema de control de concurrencia multiversión (MVCC)

Cada vez que se modifica un dato Q, no se reemplaza: se crea una nueva versión. Cuando otra transacción quiere leer Q, el sistema elige qué versión mostrar según el orden y momento en que empezó la transacción.

Ordenación por Marcas Temporales Multiversión

Cada transacción recibe una marca temporal única antes de comenzar.
El sistema utiliza esas marcas para decidir qué versiones de los datos puede leer o escribir:

- **En lecturas, la transacción obtiene la versión más reciente anterior a su marca temporal, manteniendo el orden lógico.**
- **En escrituras, si una transacción intenta modificar una versión que ya fue leída por otra más reciente, se aborta para evitar inconsistencias.**

Ventajas:

- **Las lecturas nunca fallan ni esperan.**
- **Ideal para bases donde se lee más de lo que se escribe.**

Desventajas:

- **Cada lectura actualiza su marca temporal, implicando más accesos al disco.**
- **Los conflictos se resuelven con retrocesos (abortos) en lugar de esperas, lo cual puede ser costoso**

Bloqueo de dos fases multiversión:

- **Hay transacciones de solo lectura y de actualización.**

- **Las de actualización mantienen todos los bloqueos hasta el final; se ordenan por su commit y cada dato usa una marca basada en contador (no reloj).**

- **Las de solo lectura toman su marca leyendo el contador actual y leen siguiendo el esquema de marcas temporales (no esperan).**

- **Al escribir: toman bloqueo exclusivo, crean nueva versión con marca ∞; al commit se setea la marca a contador+1 (solo una transacción de actualización confirma a la vez). Quien empiece después ve los nuevos valores; quien ya estaba antes ve los anteriores. Las Lecturas no esperan bloqueos.**

Control de concurrencia en PostgreSQL

- **Usa MVCC, donde cada consulta ve una instantánea de los datos.**

- **Varias transacciones pueden trabajar a la vez sin interferirse.**

- **Lectura y escritura no se bloquean entre sí, incluso con el nivel más estricto (Serializable Snapshot Isolation – SSI).**

- **También permite usar bloqueos por tabla o fila cuando se necesita un control manual.**

Fenómenos

Estos son problemas generados por la posible concurrencia de las transacciones

- **Lectura no confirmada o sucia (dirty read):una transacción lee datos escritos por otra transacción concurrente que no confirmó, es decir, la transacción está trabajando con datos temporales.**
- **Lectura no repetible (non repeatable read): Una transacción lee dos veces el mismo dato y encuentra valores distintos porque otra transacción lo modificó mientras tanto.**
- **Lectura fantasma (phantom read): Una transacción vuelve a ejecutar una consulta y aparecen o desaparecen filas nuevas, insertadas por otra transacción concurrente.**
- **Anomalía de serialización (serialization anomaly): Dos transacciones se ejecutan en distinto orden, modificando los mismos datos(condición de carrera).**

Niveles de aislamiento:

Read (Un)Committed Isolation Level

- **Es el nivel de aislamiento predeterminado en PostgreSQL.**
- **Cada consulta SELECT ve una snapshot de la base en el momento en que empieza.**
- **No ve los cambios de otras transacciones hasta que se confirman (COMMIT).**
- **Pero sí puede ver los cambios dentro de su misma transacción.**
- **Si dos transacciones se ejecutan al mismo tiempo, la segunda espera a que la primera termine antes de modificar los mismos datos.**
- **Evita lecturas sucias, pero puede haber lecturas no repetibles o fantasmas.**

Repeatable Read Isolation Level

- **Solo muestra datos confirmados antes de que empiece la transacción.**
- **No ve cambios de otras transacciones, pero sí los propios aún no confirmados.**
- **En PostgreSQL evita todos los fenómenos salvo las anomalías de serialización.**
- **A diferencia de Read Committed, siempre ve los mismos datos.**
- **En otros motores puede haber fantasmas, pero en PostgreSQL no.**

Serializable Isolation Level

- **Este nivel de aislamiento es el nivel más estricto emulando las transacciones serializables como si se ejecutaran una detrás de la otra.**
- **Ninguno de los fenómenos se produce cuando se utiliza este nivel de aislamiento**

Nivel de aislamiento vs. fenómeno

Niveles de aislamiento y los fenómenos que pueden producirse en cada uno.

Nivel/Fenómeno	Dirty read	Non repeatable read	Phantom read	Serialization anomaly
Read uncommited	Permitido, pero no en Postgres	SI	SI	SI
Read commited	NO	SI	SI	SI
Repeatable read	NO	NO	Permitido, pero no en Postgres	SI
Serializable	NO	NO	NO	NO

Utilización de transacciones:
comandos:
BEGIN; para empezar una transacción
COMMIT; para confirmar la transacción
ROLLBACK; para deshacer la transacción

Unidad Normalización

que es la normalización?

son pasos o formas normales que mejora la base de datos las cuales tiene como objetivo evitar la duplicidad de datos y eliminar la redundancia(tambien se puede decir que es una mejor organización de datos y mejor aprovechamiento del disco)

Anomalías comunes antes de normalizar

1. **Anomalía de INSERT: cuando se inserta un registro en un tabla hija sin que tenga relacion con la tabla padre**

2. **Anomalía de DELETE: al borrar un registro de una tabla padre dejando huerfanos a otros registros**

3. **Anomalía de UPDATE: debe considerarse la tabla padre e hija en una operacion para evitar huerfanos.**

1FN (Primera Forma Normal)

- **Se eliminan los grupos de campos repetidos creando nuevas tablas con relación 1–N.**

- **Cada campo debe tener un solo valor.**

- **Todos los registros deben poder identificarse por una clave primaria.**

2FN (Segunda Forma Normal)

- **Requiere estar en 1FN.**

- **Busca eliminar los valores repetidos o estáticos y pasar esos datos a nuevas tablas.**

- **las nuevas tablas, obtienen una clave primaria conformada por un único campo.**
- **Se eliminan las dependencias parciales**

3FN (Tercera Forma Normal)

- **Requiere estar en 2FN.**

- **Se eliminan las dependencias transitivas (un campo no debe depender de otro que no sea la clave).**

- **Si hay relaciones N–M, se crea una tabla intermedia.**
- **Se quitan campos comunes o calculados para evitar duplicaciones.**

BCNF (Forma Normal Boyce–Codd)

- **Requiere estar en 3FN.**
- **Se separan las claves candidatas en nuevas tablas.**
- **Si hay una sola clave candidata, 3FN y BCNF son iguales.**
- **Con varias claves candidatas, se dividen las tablas, aunque puede bajar el rendimiento.**

Cuarta Forma Normal (4FN)

- **Requiere estar en 3FN o BCNF.**
- **Elimina las dependencias de valores múltiples convirtiendolas en dependencias funcionales**
- **Se aplica cuando una tabla tiene campos con varios valores no relacionados(,)**
- **Esos campos deben pasarse a nuevas tablas y relacionarse con la original (relaciones 1–N o N–M).**
- **La 4FN separa conjuntos de datos para que cada registro sea más fácil de acceder y buscar.**

5FN (Quinta Forma Normal o Proyección–Unión)

- **Requiere estar en 4FN.**
- **Elimina dependencias cíclicas y redundancias(cuando la pk está compuesta por 3 o más campos y esos campos están relacionados de a pares entre sí)**
- **Se separa la tabla en varias partes (proyecciones), que al unirse muestran la misma información original.**

6FN (Forma Normal DKNF)

- **Representa un modelo totalmente normalizado.**
- **No hay anomalías de inserción, eliminación o modificación.**
- **Cada campo depende directamente de la clave primaria.**
- **No se usa mucho porque reduce el rendimiento en sistemas reales.**

Unidad Funciones y Triggers

¿Que es una función?
son aquellas que encapsulan lógica para realizar operaciones dentro de la base de datos.
Permiten obtener resultados al ser invocadas, recibir parámetros y devolver valores.

Ventajas:

- **Reutilización del código.**
- **Mayor rendimiento y homogeneidad.**
- **Posibilidad de ser usadas por distintas aplicaciones directamente.**

PostgreSQL permite escribir funciones en varios lenguajes:
SQL, PL/pgSQL, C, Python, etc

sintaxis

```
CREATE FUNCTION nombre_funcion(parametros)
RETURNS tipo_retorno AS $$
BEGIN
   -- instrucciones
END;
$$ LANGUAGE plpgsql;
```

Sintaxis

```
CREATE OR REPLACE FUNCTION <nombre>([[<mode>] <arg1> <type>, ...,[<mode>]
<argN> <type>]) RETURNS <ret_type> AS $$
DECLARE <var_name> <var_type>
BEGIN
...;
END;
$$ LANGUAGE <language>;
```

- **mode**: IN|OUT|INOUT
- **type**: tipo de cada argumento. Si el tipo está basado en alguna columna, se puede utilizar la siguiente expresión: <table_name>.<column_name>%TYPE
- **ret_type**: tipo de retorno de la función. Puede ser un tipo básico o TABLE, en este caso hay que especificar las columnas y sus tipos.
- **language**: puede ser sql; plpgsql; plpythonu

Tipos de funciones

- **SQL: solo permiten sentencias SQL simples.**
- **PL/pgSQL: permiten estructuras de control, bucles, condiciones, etc.**
- **PL/PythonU: se programan en Python con todo su potencial.**

```
-- Ejemplo de función que permite actualizar un registro de una tabla
CREATE FUNCTION tf1 (accountno integer, debit numeric) RETURNS integer AS $$
  UPDATE bank
    SET balance = balance - debit
    WHERE accountno = tf1.accountno;
  SELECT 1;
$$ LANGUAGE SQL;


-- Ejemplo de función cuyo tipo de retorno es una tabla con 2 campos
CREATE FUNCTION dup(int) RETURNS TABLE(f1 int, f2 text) AS $$
  SELECT $1, CAST($1 AS text) || ' is text'
$$ LANGUAGE SQL;


SELECT * FROM dup(42);

-- Genera la siguiente salida
 f1 |     f2
----+------------
 42 | 42 is text
(1 row)
```

en este ejemplo se ve que se usan practicas como select 1 que indicaria que la funcion se ejecuto correctamente

y en la segunda se usa como retorno una tabla con varios campos

y tambien el select $1 que indicaria el primer parámetro de la funcion y el cast sabemos que sirve para castear un dato a otro tipo

```
-- Ejemplo de función que valida un par de credenciales
CREATE OR REPLACE FUNCTION check_password(uname TEXT, pass TEXT) RETURNS BOOLEAN AS
$$
DECLARE passed BOOLEAN;
BEGIN
  passed := false;
  SELECT (password = $2) INTO passed
  FROM cliente
  WHERE name = $1;

  -- en caso de que el query no retorne filas, la variable passed contiene NULL.
  IF passed IS NULL THEN
      passed := false;
  END IF;

  RETURN passed;
END;
$$ LANGUAGE plpgsql;
```

```
-- Ejemplo de función que suma 2 enteros si y sólo si ambos son positivos
CREATE FUNCTION sum_positive(n1 integer, n2 integer) RETURNS integer AS $$
DECLARE sum integer;
BEGIN
  IF(n1 > 0 AND n2 > 0) THEN
    RETURN n1+n2;
  ELSE
    RAISE NOTICE 'Error!!';
  END IF;
END;
$$ LANGUAGE plpgsql;
-- Ejemplo de función que devuelve el registro con el id mayor.
CREATE OR REPLACE FUNCTION last_record() RETURNS provincias AS $$
DECLARE  t2_row provincias%ROWTYPE;
BEGIN
       select * into t2_row
       from provincias order by prv_id desc limit 1;

       return t2_row;
END;

$$ LANGUAGE plpgsql;
```

Excepciones

Permiten manejar errores dentro de una función.

```
RAISE [nivel] [mensaje]
```

- **Niveles: DEBUG, LOG, INFO, NOTICE, WARNING, EXCEPTION (este último detiene la ejecución).**
- **Se puede personalizar el mensaje y configurar qué nivel registrar en el archivo postgresql.conf.**

Ejemplo de exepción

```
-- Ejemplo de función que valida un par de credenciales
CREATE OR REPLACE FUNCTION check_password(uname TEXT, pass TEXT) RETURNS BOOLEAN AS
$$
DECLARE passed BOOLEAN;
BEGIN
  passed := false;
  SELECT (password = $2) INTO passed
  FROM cliente
  WHERE name = $1;

  -- en caso de que el query no retorne filas, la variable passed contiene NULL.
  IF passed IS NULL THEN
    RAISE LOG 'El usuario % no existe', uname;
      passed := false;
  END IF;

  RETURN passed;
END;
$$ LANGUAGE plpgsql;


-- Al ejecutar la función
select check_password('juan', 'clave_1234');

-- Obtenemos la salida
 check_password
----------------
 f
(1 row)


-- Y si vemos el archivo de log, veremos el siguiente mensaje
2021-10-28 03:45:13.093 UTC [49] LOG:  El usuario juan no existe
2021-10-28 03:45:13.093 UTC [49] CONTEXT:  PL/pgSQL function
check_password(text,text) line 10 at RAISE
2021-10-28 03:45:13.093 UTC [49] STATEMENT:  select check_password('juan',
'clave_1234');
```

Triggers

Un trigger (disparador) es una instrucción que hace que el motor de base de datos ejecute una función automáticamente cuando ocurre un evento específico sobre una tabla.
El tipo de retorno de la funcion tiene que ser trigger

Tipos de triggers

- **BEFORE: se ejecuta antes del evento**

- **AFTER: se ejecuta después del evento**

When	Event	Row-level	Statement-level
Before	insert/update/delete	Tables	Tables + Views
	truncate	-	Tables
After	insert/update/delete	Tables	Tables + Views
	truncate	-	Tables

Registros especiales
cuando ejecuta un trigger, el sistema inyecta automáticamente registros especiales dentro de la función asociada al trigger.

- **NEW:**
  - **contiene los datos nuevos (para INSERT o UPDATE).**
  - **Solo existe en triggers a nivel de fila**
  - **Es NULL en operaciones de delete o para triggers que se ejecutan por operación.**

- **OLD:**
  - **contiene los datos anteriores (para DELETE o UPDATE).**

- **Solo existe en triggers a nivel de fila**
- **Es NULL en operaciones insert o update o para triggers que se ejecutan por operación.**

Creación de triggers

```
CREATE [ CONSTRAINT ] TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event [ OR
... ] }
ON table
[ FROM referenced_table_name ]
[ NOT DEFERRABLE | [ DEFERRABLE ] { INITIALLY IMMEDIATE | INITIALLY
DEFERRED } ]
[ REFERENCING { { OLD | NEW } TABLE [ AS ] transition_relation_name } [ ... ] ]
[ FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( condition ) ]
EXECUTE PROCEDURE function_name ( arguments )
```

- PostgreSQL admite especificar una condición booleana en la definición del trigger sobre la cual basa su decisión para ejecutar el trigger o no. Esta condición se especifica mediante la palabra ***WHEN***
- **Nota**: si se definen varios triggers sobre la misma tabla y para el mismo evento, los mismos se ejecutarán en orden alfabético.
- **Nota II**: puede existir recursividad en un trigger. Es responsabilidad del programador que no se creen loops infinitos.

Ejemplos

```
CREATE TRIGGER check_update
BEFORE UPDATE ON accounts
FOR EACH ROW
EXECUTE PROCEDURE check_account_update();
```

```
CREATE TRIGGER check_update
BEFORE UPDATE ON accounts
FOR EACH ROW
WHEN (OLD.balance IS DISTINCT FROM NEW.balance)
EXECUTE PROCEDURE check_account_update();
```

```
CREATE OR REPLACE FUNCTION before_insert_provincias() RETURNS trigger AS $$

BEGIN
      RAISE INFO 'ejecuta trigger';
      IF length(new.prv_nombre) < 3 THEN
         return null;
      END IF;
      return new;
END;

$$ LANGUAGE plpgsql;


CREATE TRIGGER before_insert_provincias
BEFORE INSERT ON provincias
FOR EACH ROW
EXECUTE PROCEDURE befor_insert_provincias();
```

Unidad Índices y Optimización de Consultas

que es un indice?

Un índice es una estructura que almacena datos copiados de uno o más campos de una tabla para acelerar las búsquedas y el acceso a registros de esta forma.

Ventajas:

- **Acelera consultas SELECT con condiciones**
- **Permite localizar registros de forma más eficiente.**

Desventajas:

- **Cada actualización a una tabla (crear, modificar o eliminar un registro) impactará en los índices que la misma posea**

sintaxis:

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ name ] ON table [ USING method ]
( { column | ( expression ) }[ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] )
[WHERE predicate]
```

CREATE INDEX nombre_idx
ON tabla (columna);

tambien existen otras opciones :

- **CONCURRENTLY: crea el índice sin bloquear la tabla.**
- **METHOD: método de índice (b tree, hash, gin, gist, spgist, brin).**
- **EXPRESSION: permite usar funciones o expresiones (lower(nombre)).**
- **PREDICATE: define un índice parcial (solo sobre ciertas filas).**

Funcionamiento

El procesador de consultas decide si usa un índice o recorre toda la tabla:

- **Si hay pocos registros, es más rápido un acceso secuencial.**
- **Si hay muchos registros, conviene usar un índice.**
- **Cada actualización impacta en todos los índices de esa tabla, afectando el rendimiento.**

Heap:
El heap es el espacio físico de memoria o disco donde se guardan los registros de una tabla.
Los índices solo apuntan a la posición de esos registros dentro del heap.

Tipos de indices:

- **GIST:**
  - **Ideal para indexar datos complejos que no se “adaptan” a un índice btree**
  - **Permite indexar datos geográficos, arrays, par clave/valor**

- **GIN:**
  - **Excelente para valores compuestos o repetidos (texto, JSON, arrays).**
  - **Guarda pares (clave, valor) con los IDs de las filas donde aparece cada valor.**
- **Expression index:**
  - **Permite definir el índice a partir de una expresión, función, concatenación, etc.**
- **indices parciales (Partial Index)**
  - **Cubre solo parte de la tabla según una condición (WHERE state = 'AZ'), reduciendo tamaño y sobrecarga.**

Clustering

Es reordenar los datos de una tabla de acuerdo a un índice creado previamente y reescribirla en disco.

```
CLUSTER [VERBOSE] table_name [ USING index_name ]
```

Ejemplo

```
CREATE INDEX geom_idx ON TABLE my_tabla_geo USING GIST(the_geom);

CLUSTER my_tabla_geo USING geom_idx;
```

Optimización de consultas

SQL es un lenguaje declarativo, es decir: le decimos qué queremos, pero no cómo hacerlo.
Por eso el motor de PostgreSQL analiza cada consulta y elige el plan de ejecución más eficiente.

El comando EXPLAIN permite ver, evaluar y mejorar el plan de ejecución.

```
EXPLAIN select * from table_dummy;

EXPLAIN VERBOSE select * from table_dummy;
```

Comando EXPLAIN

Muestra cómo la base de datos planea ejecutar una consulta.
Cada plan se representa como un árbol de ejecución, con nodos leídos de abajo hacia arriba.
Cada nodo muestra:

- **Costo estimado para empezar y terminar la consulta.**
- **Cantidad estimada de filas y bytes procesados.**

```
QUERY PLAN
-------------------------------------------------------------
Seq Scan on table_dummy (cost=0.00..458.00 rows=10000 width=244)
```

lo que el EXPLAIN terminara evaluando en su plan sera :

- **Scan Methods (sequential, index, bitmap index)**
- **Join Methods (nested loops, hash join, merge join)**
- **Aggregation Methods (hash, sort)**

Métodos de escaneo (Scan Methods)
Formas de leer los datos de las tablas:

Método	**Descripción**
Sequential Scan	**Recorre toda la tabla (más rápido si el resultado incluye más del 40 % de los registros).**
Index Scan	**Usa un índice para acceder solo a las filas necesarias.**
Bitmap Index Scan	**Combina varios índices o condiciones para optimizar la búsqueda.**

Métodos de unión (Join Methods)
son aquellos que utilizan el motor de base de datos para unir/cruzar los datos de las tablas.

Tipo de Join	**Descripción**
Nested Loop	**Compara cada fila de una tabla con cada fila de la otra. Solo eficiente con tablas pequeñas o índices.(sequential o con index)**
Hash Join	**Crea una tabla hash en memoria con las claves de una tabla y compara con la otra. Muy común y rápido si la tabla cabe en la memoria.**
Merge Join	**Ordena ambas tablas y las recorre en paralelo. Ideal para tablas grandes o ya ordenadas por índice.**

Comandos útiles

- **ANALYZE: ayuda a recolectar estadísticas sobre la distribución de los datos**

- **VACUUM: limpia índices y tablas.**